

Failure (bad key + cost comparison) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Work through in order — don't jump ahead until you've genuinely tried.

- [Hint 1](#)
- [Hint 2](#)
- [Hint 3](#)
- [Hint 4](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 1

Simple Version

Two separate things to try here: 1. Use a fake API key on purpose and see exactly what error comes back. 2. Ask the model the same question twice — once with a short system prompt, once with a long one — and compare how many tokens each one used.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

Part 1 tests that you catch `openai.AuthenticationError` *specifically*, not just any exception — because a bare `except Exception` would also hide a real bug in your own code behind the same generic message, making it much harder to tell "bad key" apart from "I broke something."

Part 2 uses `response.usage` — every response object carries back the exact token counts OpenAI billed you for, split into prompt tokens and completion tokens. Comparing this number between a 1-sentence and a 20-sentence system prompt turns "shorter prompts cost less" from a rule you were told into a number you measured yourself.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 2

Simple Version

- `OpenAI(api_key="fake-key-123")` — make a client with a wrong key on purpose.
- Wrap the call in `try/except`, catching the specific auth error, not a bare `except`.
- `response.usage` — has the token counts on it after any real, successful call.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

- `OpenAI(api_key="sk-fake-key-123")` — passing a bad key directly to the client constructor (instead of relying on the real one from `.env`) forces an authentication failure on the very next call.
- `except openai.AuthenticationError as e:` — this is the specific exception type raised for a bad/missing key, distinct from `RateLimitError`, `BadRequestError`, etc. Catching it specifically is what lets you print a clean, correct message ("check your API key") instead of a generic one that could mean anything.
- `response.usage.prompt_tokens`, `response.usage.completion_tokens`, `response.usage.total_tokens` — every successful response carries these back, so you don't need `tiktoken` here at all; just read what the API already tells you it charged for.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 3

Simple Version

part 1:

- make a client with a fake key
- try to send a message
- except the specific "bad key" error:
 - print a clean message

part 2:

- ask the same question with a short system prompt, note the token count
- ask the same question with a long system prompt, note the token count
- compare the two numbers

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
part 1:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(...)
    except openai.AuthenticationError as e:
        print a clean message, not the raw exception

part 2:
    short_response = client.chat.completions.create(system="Be helpful.", ...)
    long_response = client.chat.completions.create(system="<20-sentence prompt>", ...)
    print short_response.usage.total_tokens
    print long_response.usage.total_tokens
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 4

Simple Version

```
import openai
from openai import OpenAI

bad_client = OpenAI(api_key="sk-fake-key-123")
try:
    bad_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "hi"}],
    )
except openai.AuthenticationError:
    print("Login failed – check your API key.")

# now write part 2 yourself: two calls, one short system prompt, one long, compar
```

Intermediate Version

```
import openai
from openai import OpenAI

client = OpenAI()
bad_client = OpenAI(api_key="sk-fake-key-123")

try:
    bad_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "hi"}],
    )
except openai.AuthenticationError:
    print("Login failed – check your API key.")

# now write the cost comparison yourself, then check the Solution
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Full solution: [Show me the solution](#)